

Foundation Models and Generative Artificial Intelligence

ARIN 5203

Instructor: Dr. Qifeng Chen

Course

Website

- <https://cqf.io/ARIN5203>

Venue:

- Room 5404

Time

- Monday 7:30 – 10:20 PM
- A break of about 10 minutes in between

Who we are

Instructor: Qifeng Chen (cqf@ust.hk)

Office hours: 10:30 am Thu in Rm3519

TAs:

Guotao Meng (gmeng@connect.ust.hk)

Mingzhe Zheng (mzhengar@connect.ust.hk)

Liya Ji (jiliya@ust.hk)

Logistics

- Three Assignments (30%)
 - First assignment will be out in about two weeks
- Midterm (35%)
 - The midterm will be in class
- Presentations of generative AI projects(5%)
- Final Course Project (30%)
 - Proposal (5%) + Milestone (5%) + Final Report (15%) + Project Presentation (5%)

Syllabus

- Basics of deep learning and foundation models
- Transformer and Attention mechanism
- Variants of transformer architectures
- Foundation models: LLM, VLM, and VLA
- Generative models: VAE, normalizing flow, GAN, diffusion models
- Pre-training and post-training of foundation models
- Image, video, and audio generation
- Scaling laws, retrieval augmented generation, and alignment
- Midterm exam
- Presentations on recent generative AI research
- Course project sharing

Deep Learning

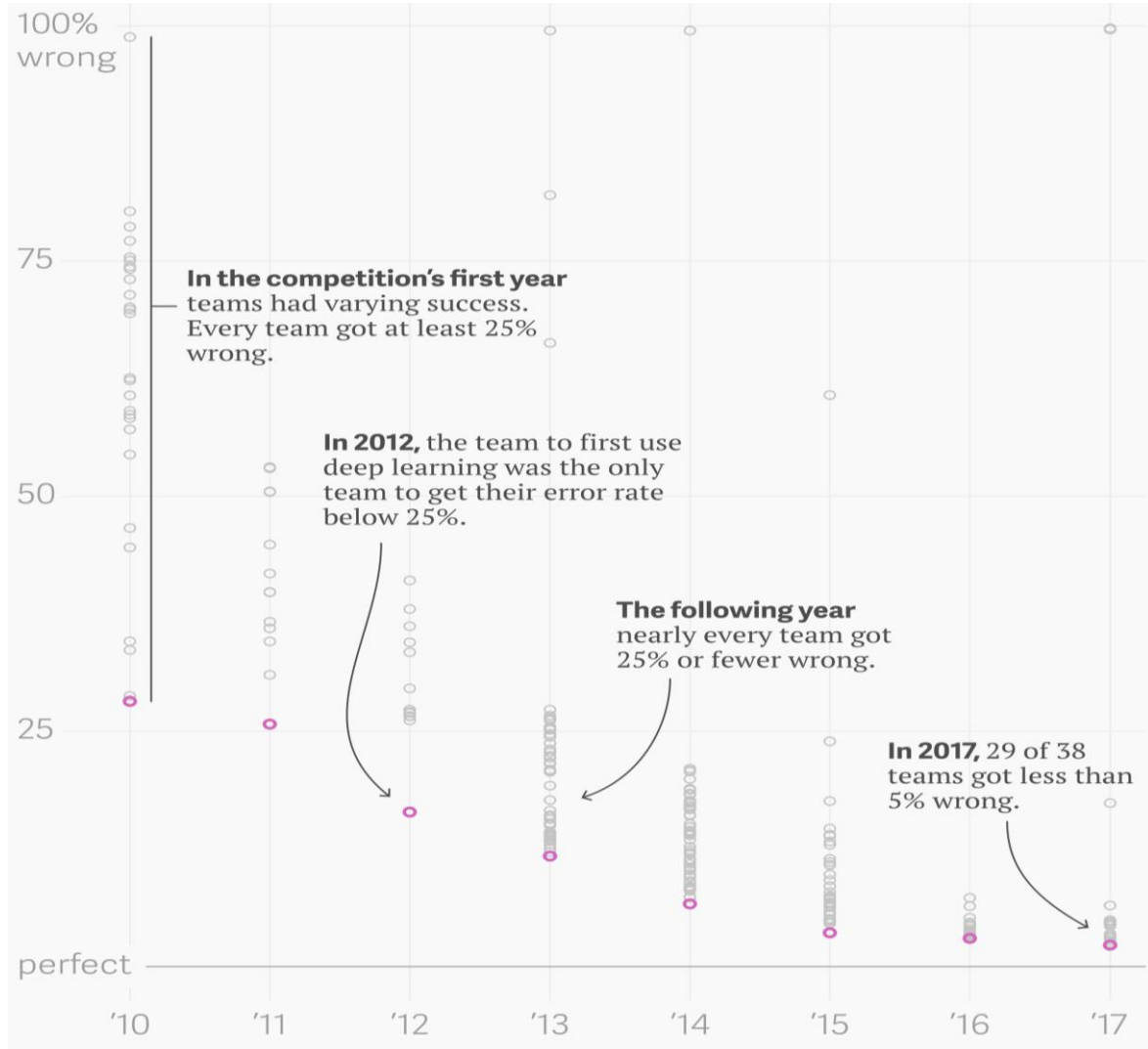


Classify Images



<http://www.image-net.org/>

Classify Images



Yanofsky, Quartz

<https://qz.com/1034972/the-data-that-changed-the-direction-of-ai-research-and-possibly-the-world/>

Detect and Segment Objects



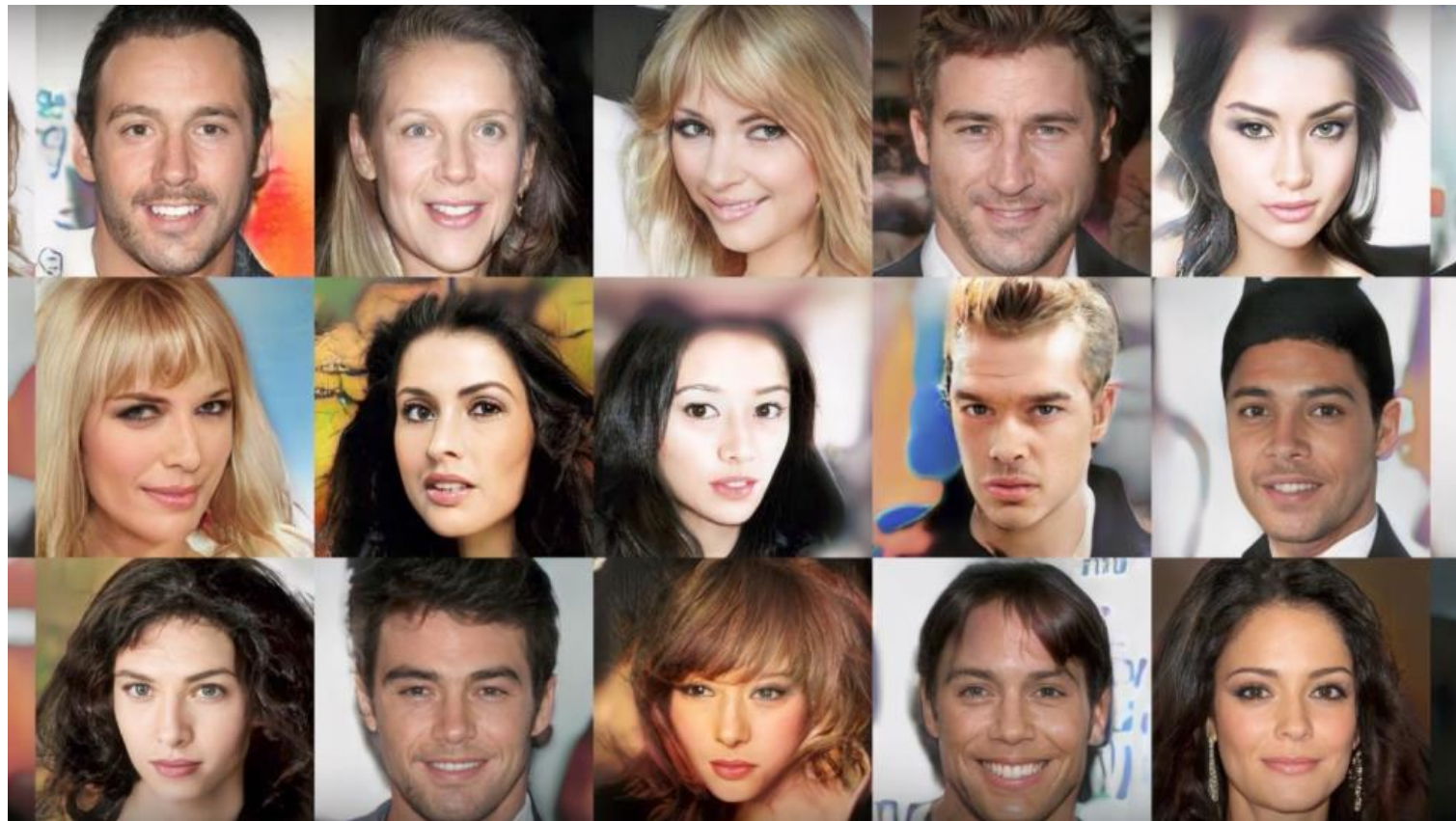
https://github.com/matterport/Mask_RCNN

Style transfer



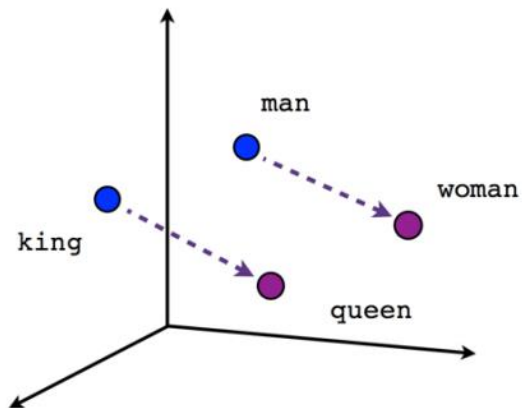
<https://github.com/zhanghang1989/MXNet-Gluon-Style-Transfer/>

Synthesize Faces

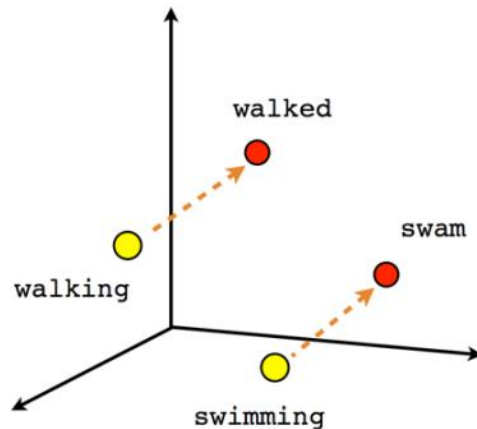


Karras et al, ICLR 2018

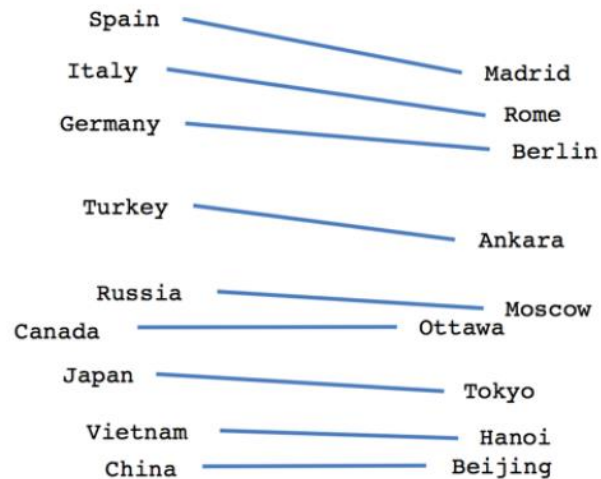
Analogies



Male-Female



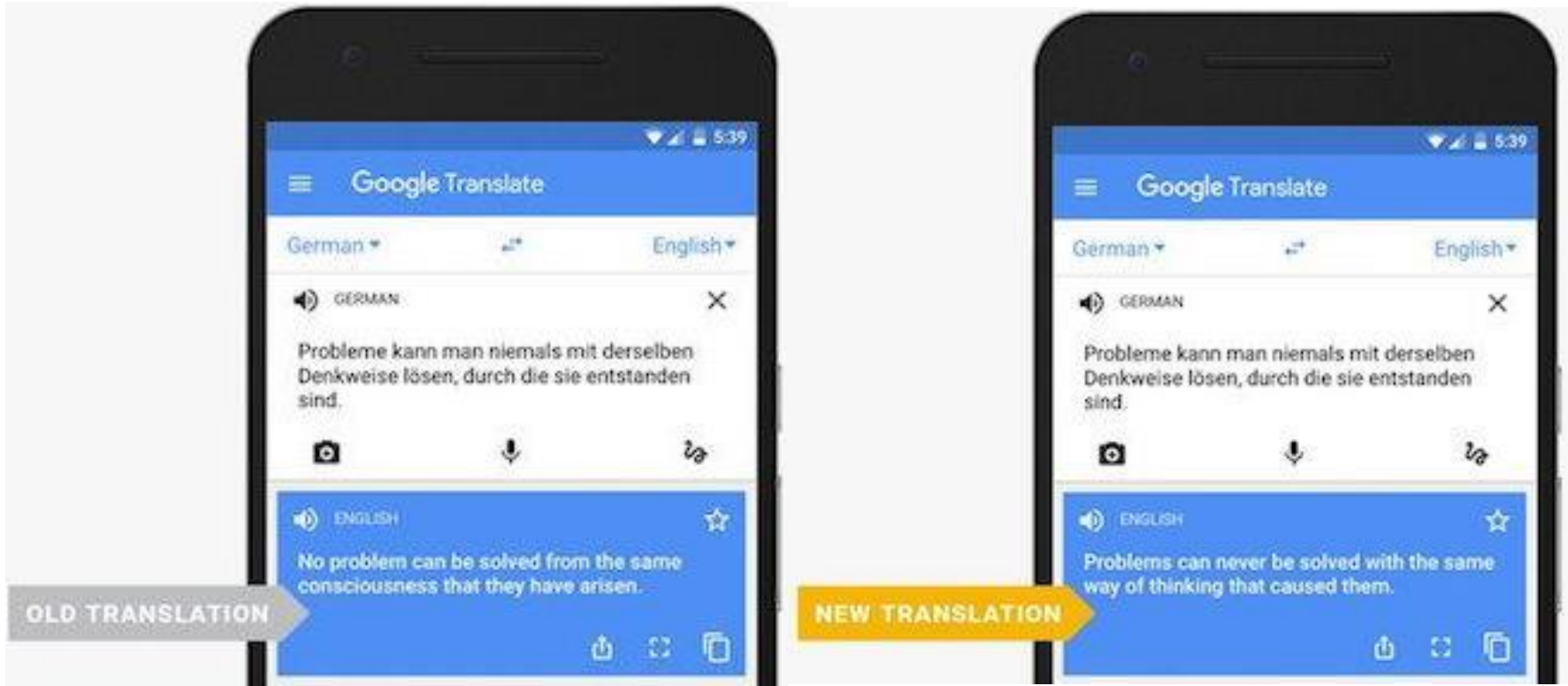
Verb tense



Country-Capital

<https://www.tensorflow.org/tutorials/word2vec>

Machine Translation



<https://www.pcmag.com/news/349610/google-expands-neural-networks-for-language-translation>

Question answering

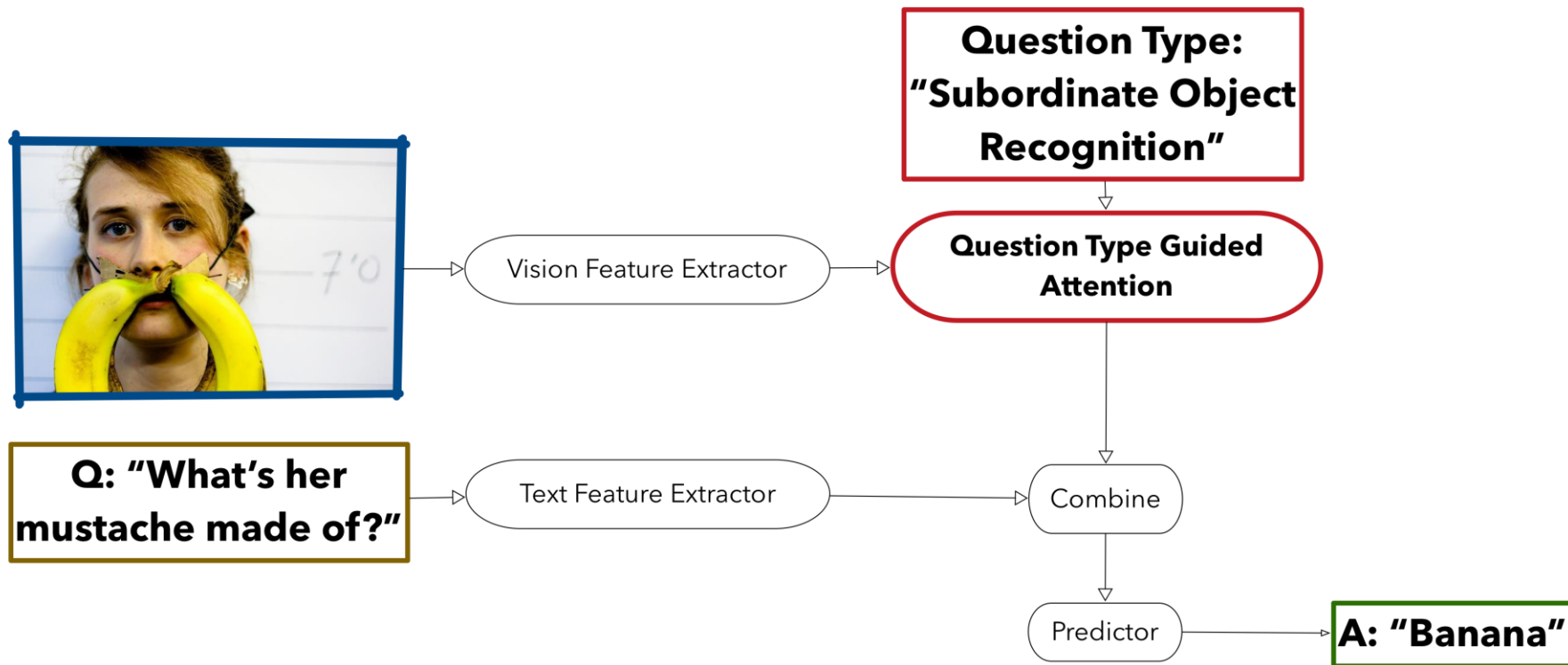
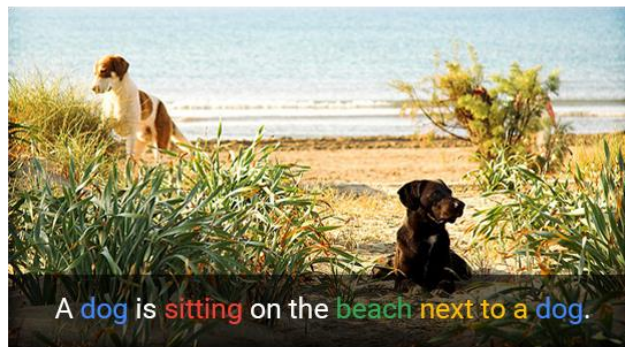


Image captioning

Human captions from the training set



Automatically captioned



Shallue et al, 2016

<https://ai.googleblog.com/2016/09/show-and-tell-image-captioning-open.html>

ChatGPT

User help me write a short note to introduce myself to my neighbor

ChatGPT Hi there,

My name is [Your Name] and I just moved in next door. I wanted to reach out and introduce myself. I'm looking forward to getting to know you and the neighborhood. If you ever need anything or just want to chat, please don't hesitate to stop by or give me a call.

Best,
[Your Name]

User can you make it more formal?

ChatGPT Dear Neighbor,

I am writing to introduce myself. My name is [Your Name] and I have recently moved in next door. I wanted to take a moment to reach out and say hello. I am looking forward to getting to know you and the neighborhood.

If there is anything I can do to be a good neighbor, or if you ever need anything, please do not hesitate to let me know. I am happy to help in any way that I can.

Sincerely,
[Your Name]

Generative AI: Text to Image

TEXT DESCRIPTION

An astronaut Teddy bears A bowl of
soup

riding a horse lounging in a tropical resort
in space playing basketball with cats in
space

in a photorealistic style in the style of Andy
Warhol as a pencil drawing

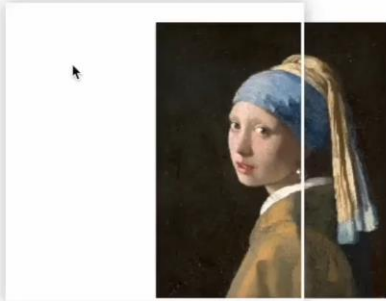


DALL-E 2



Source: Dall E 2

Generative AI: Image Completion



Generative AI: Video Generation

Text-to-Video Generation

Sora by OpenAI



Kling by Kuaishou



Veo3 by Google



Generative AI: Video Generation

Open-source Text-to-Video Generation

Hunyuan Video



Step-Video-T2V



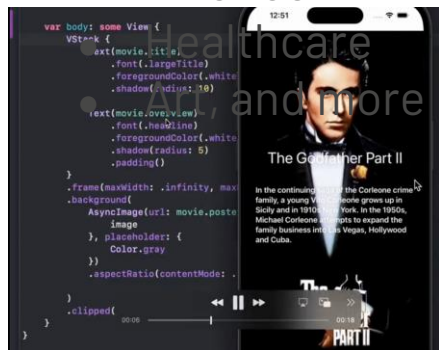
Wan Video



The Age of Generative AI!

Large Language Models (LLMs) and Generative Vision Models are rapidly evolving how we work and create:

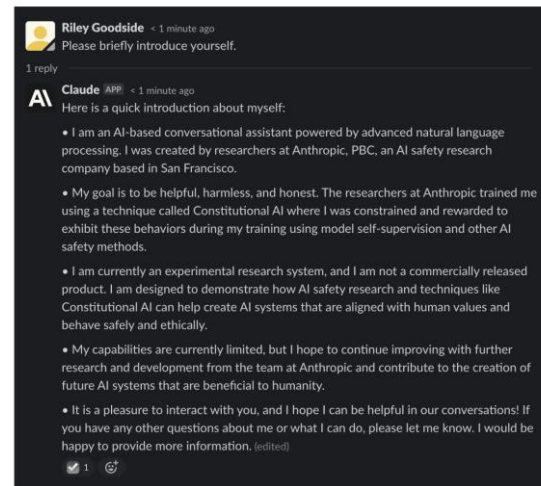
- Education
- Software Engineering
- Productivity
- Law
- Finance



GPT4 writes an iOS app



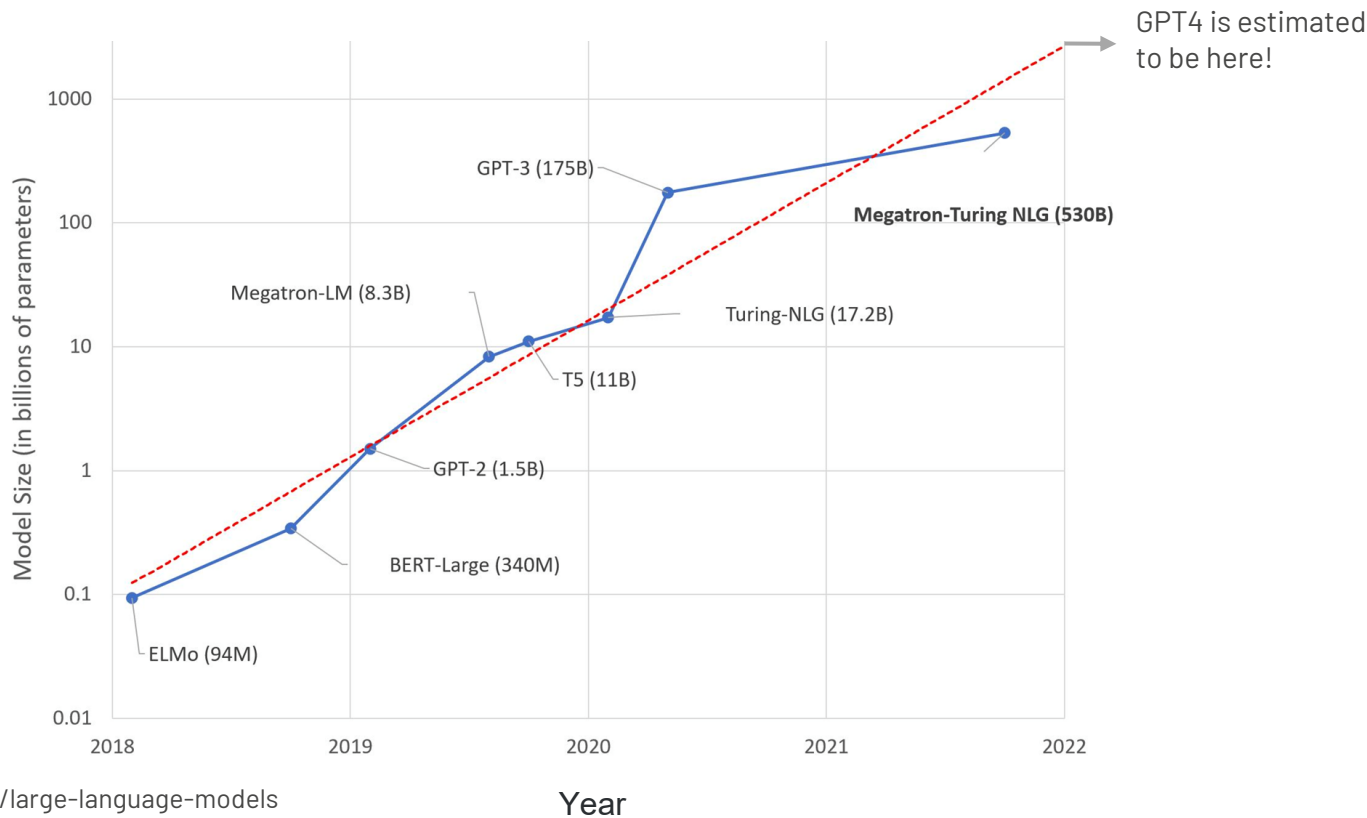
Midjourney's AI generates images



Anthropic's Claude describes its goal to be a helpful, harmless, and honest assistant



Exponential Growth of Large Language Models



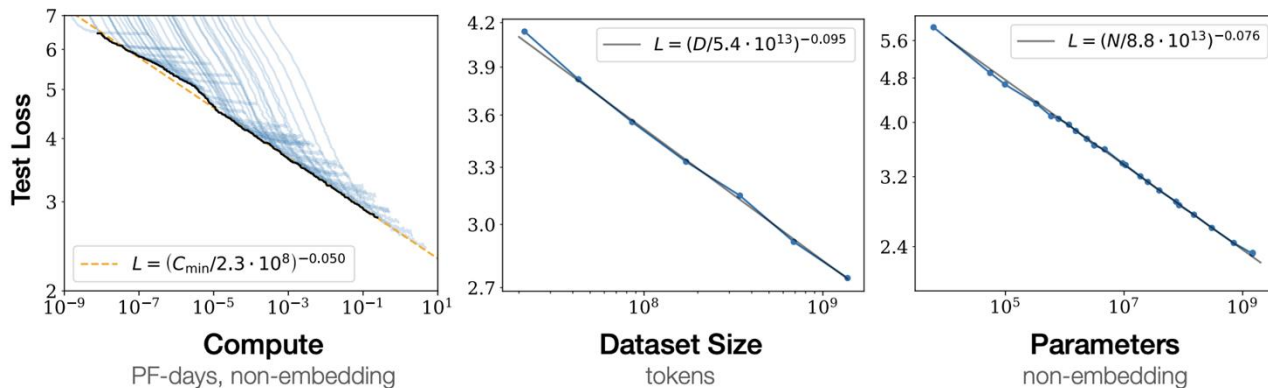
Why Do LLMs Keep Getting Bigger?

- Continuous improvement with scaling
- Emergent Behavior: Capabilities that only emerge in larger models

Improvement with Scaling

Performance of LLM continuously improve with scaling:

- Amount of compute used
- Dataset size
- Number of parameters in the model



Emergent Behavior: Zero/Few Shot Learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

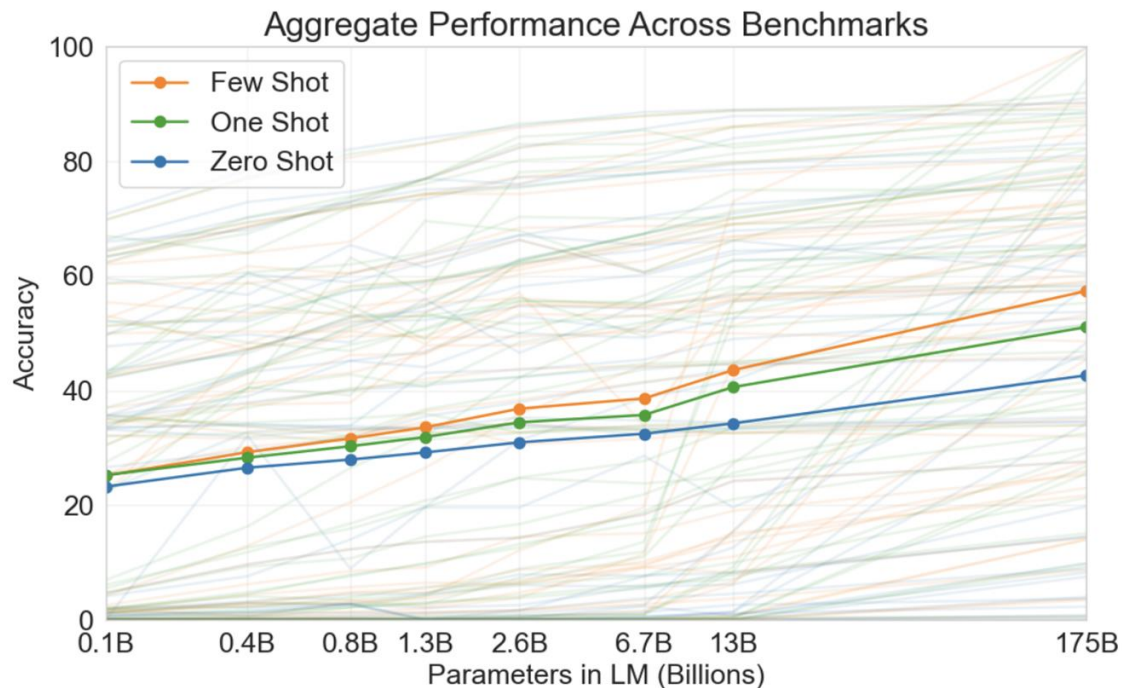
```
1  Translate English to French:  ← task description
2  cheese => .....           ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1  Translate English to French:  ← task description
2  sea otter => loutre de mer    ← examples
3  peppermint => menthe poivrée ←
4  plush girafe => girafe peluche ←
5  cheese => .....             ← prompt
```

Few Shot Learning Emerges in Larger Models



Emergent Behavior: Chain of Thought

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

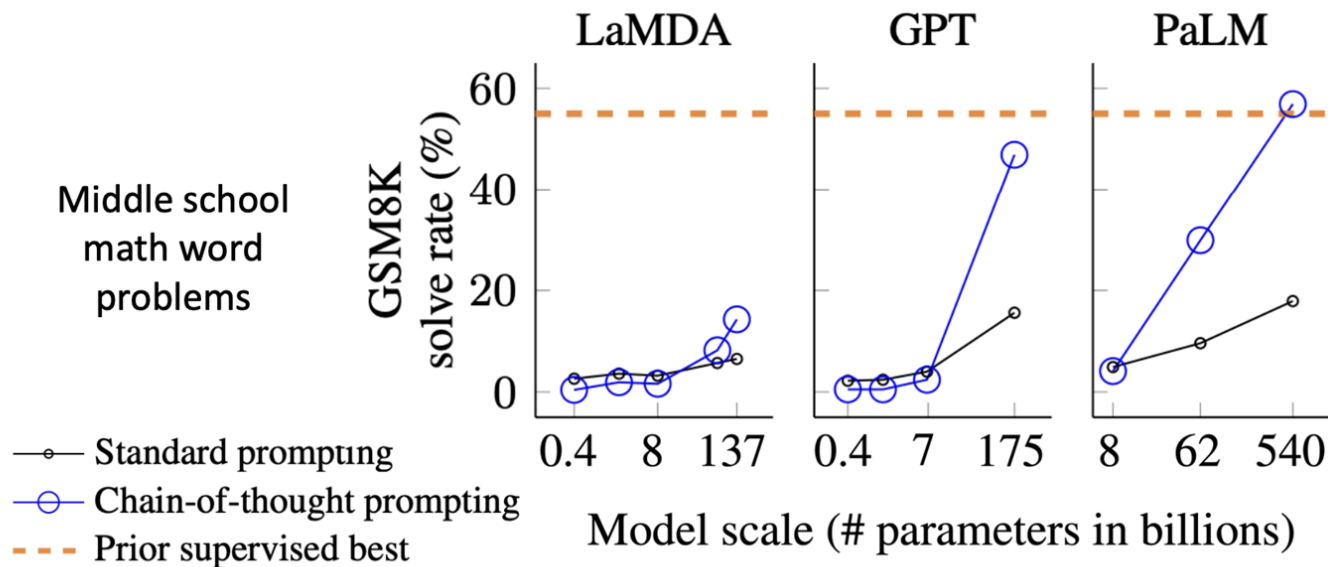
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

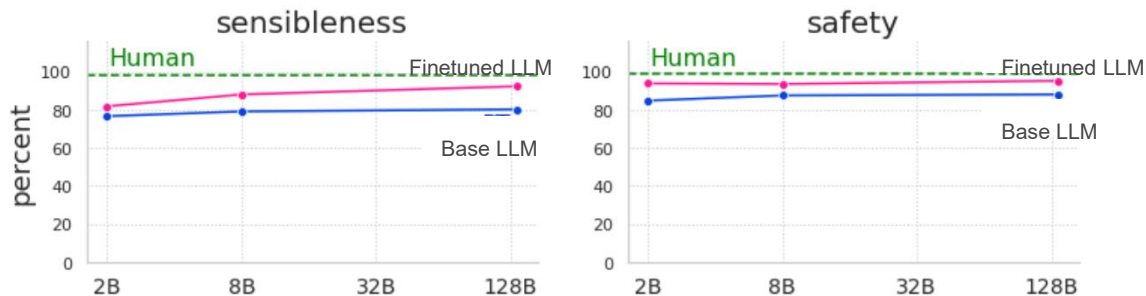
Chain of Thought Emerges in Large Models



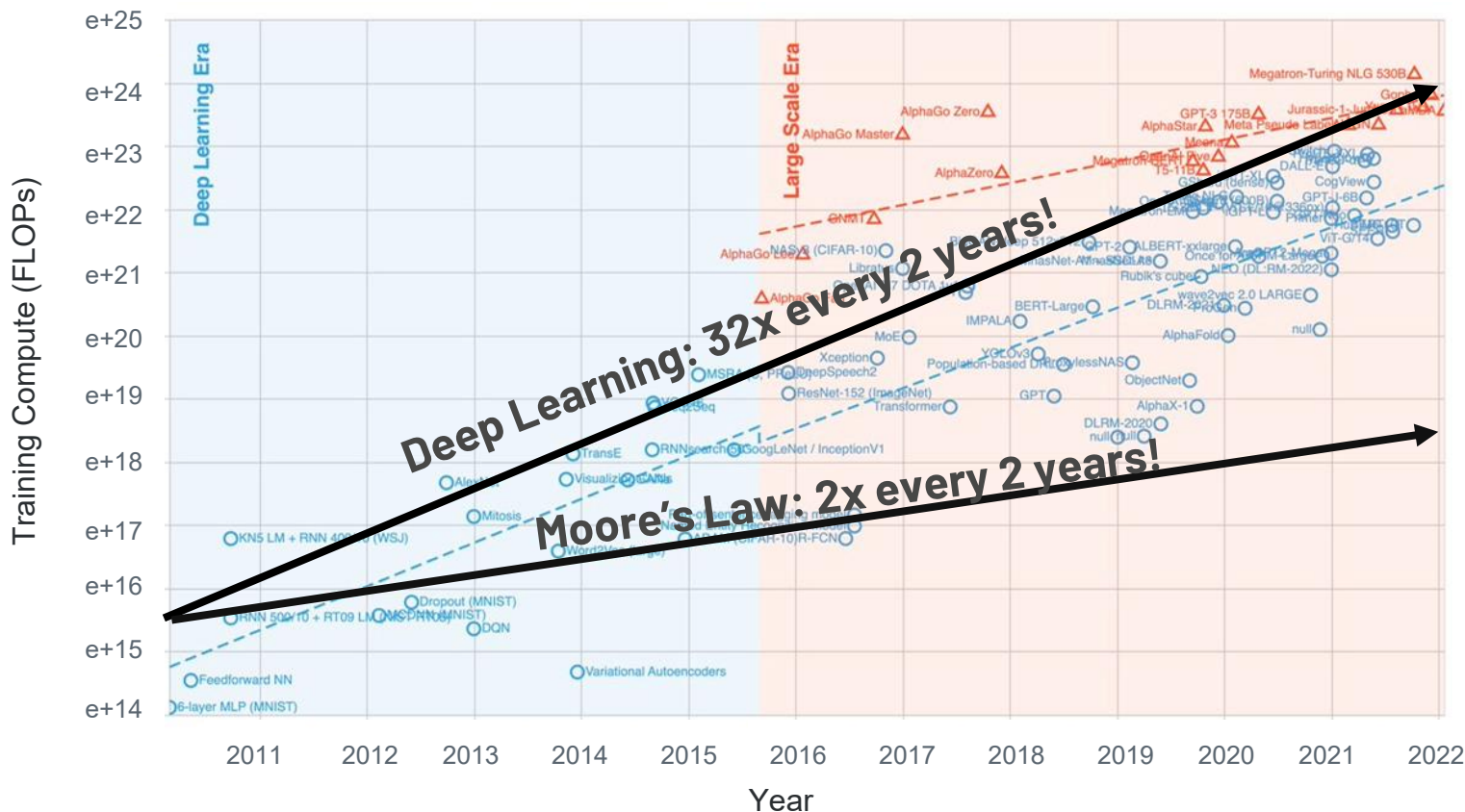
Finetuning Models Improves AI Alignment

AI Alignment is the problem of steering AI models to be aligned with intended goals, values and preferences of humans.

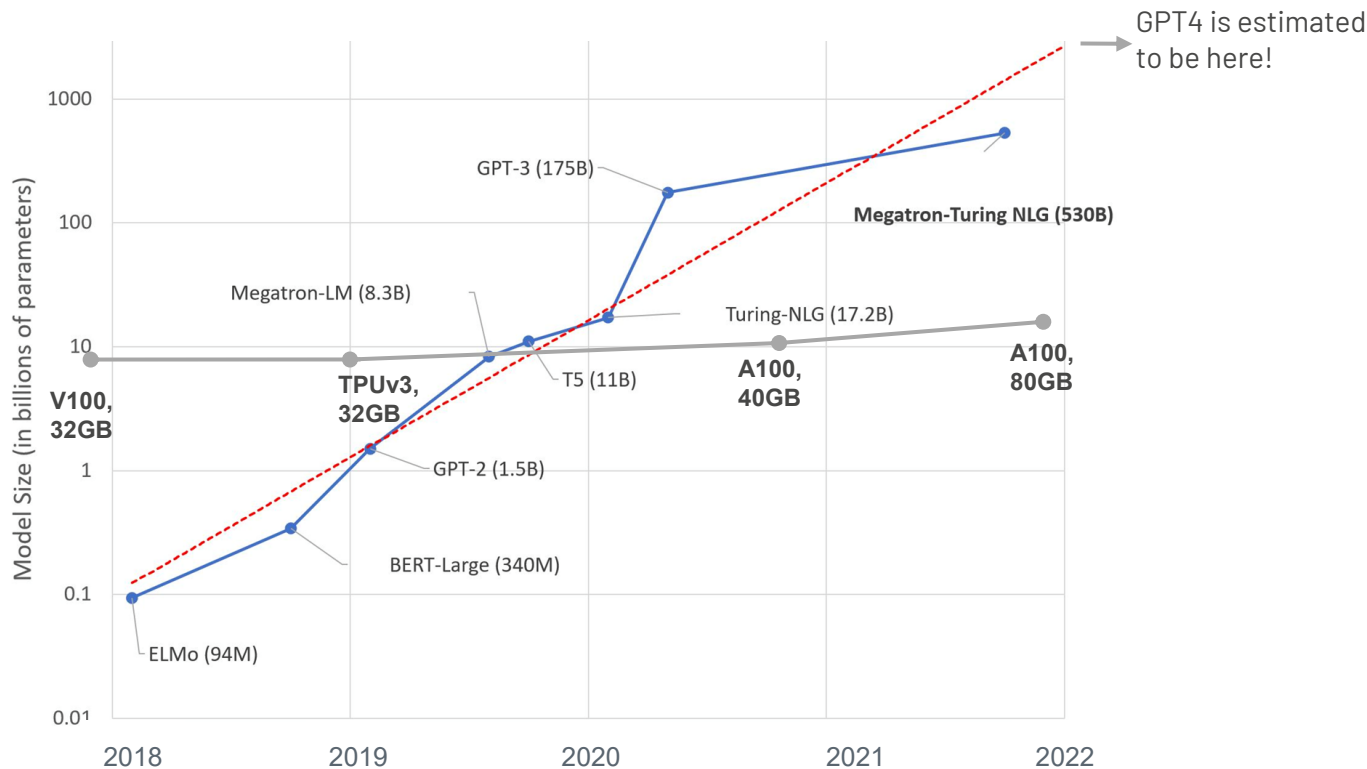
We can finetune (or update the weights of) language models to make them more aligned.



Large Computation Cost: AI models vs. Moore's Law

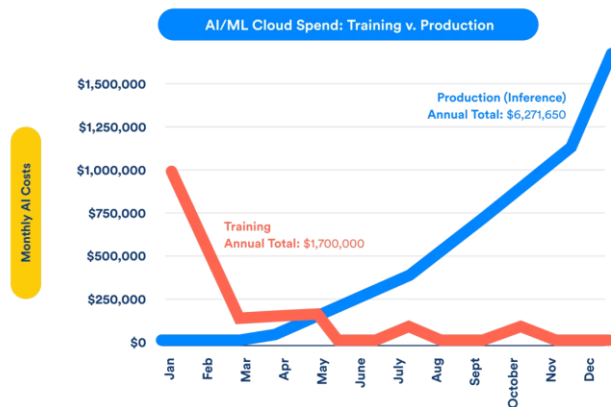


Large Memory Cost: AI models vs. Accelerator Memory



Training and Inference Costs

- Training:
 - A large upfront cost
 - For large LLMs, training could cost tens of millions of dollars and more
- Inference:
 - Everytime the model processes an input (e.g., generates a response to a prompt), it incurs an inference cost
 - A single inference call is cheap (< \$0.0001 for large LLMs), but accumulates over time



An example production model shows the large upfront cost of training and increasing cost of inference as users increase (source: OctoML)

Deep Learning Efficiency Is Extremely Important!

- Massive-scale deployment and usage in all industries
- Large models take several months to train
- Real-world applications require low-latency inference
- Large power, latency, and hardware/cloud costs of training and inference

Software



Tools

- **Python**

- Everyone is using it in machine learning & data science
- Conda package manager (for simplicity)

- **Jupyter**

- So much easier to keep track of your experiments
- Obviously you should put longer code into modules

Colab

- Go to colab.research.google.com
- Activate the GPU supported runtime
- Pro version: <https://colab.research.google.com/signup>

Pay As You Go	Recommended Colab Pro	Colab Pro+
HK\$77.80 for 100 Compute Units HK\$489.50 for 500 Compute Units You currently have 0 compute units.	HK\$77.80 per month	HK\$489.50 per month
✓ No subscription required. Only pay for what you use. ✓ Faster GPUs Upgrade to more powerful premium GPUs.	✓ 100 compute units per month Compute units expire after 90 days. Purchase more as you need them. ✓ Faster GPUs Upgrade to more powerful premium GPUs. ✓ More memory Access our higher memory machines. ✓ Terminal Ability to use a terminal with the connected VM.	✓ 500 compute units per month Compute units expire after 90 days. Purchase more as you need them. ✓ Faster GPUs Priority access to upgrade to more powerful premium GPUs. ✓ More memory Access our higher memory machines. ✓ Background execution Upgrade your notebooks to keep executing for up to 24 hours even if

SuperPOD at HKUST

HKUST SuperPOD is a state-of-the-art AI supercomputing facility

- <https://itso.hkust.edu.hk/services/academic-teaching-support/high-performance-computing/superpod>
- Each MSc student has credits to use SuperPOD





**GPUs love matrices and vectors
(they have many processors)**

A photograph of a lecture hall. In the foreground, there are several rows of wooden chairs, some with graffiti. In the background, there is a blackboard with the word 'ndarray' written on it. To the left of the blackboard is a wooden podium with a laptop on it. Above the podium is a clock. To the right of the blackboard is a door. The room has white walls and a white baseboard.

ndarray

N-dimensional Array Examples

N-dimensional array, short for ndarray, is the main data structure for machine learning and neural networks

0-d (scalar)



1.0

A class label

1-d (vector)



[1.0, 2.7, 3.4]

A feature vector

2-d (matrix)

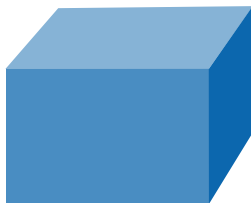


[[1.0, 2.7, 3.4]
[5.0, 0.2, 4.6]
[4.3, 8.5, 0.2]]

A example-by-
feature matrix

ND Array Examples, cont

3-d



```
[[[0.1, 2.7, 3.4]  
 [5.0, 0.2, 4.6]  
 [4.3, 8.5, 0.2]]  
 [[3.2, 5.7, 3.4]  
 [5.4, 6.2, 3.2]  
 [4.1, 3.5, 6.2]]]
```

A RGB image
(width x height
x channels)

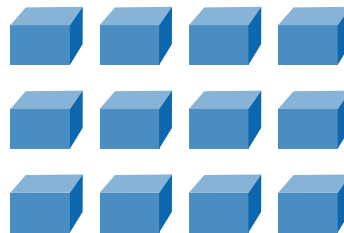
4-d



```
[[[ [...  
    ...  
    ... ]]]
```

A batch of
RGB images
(batch-size x
width x height
x channels)

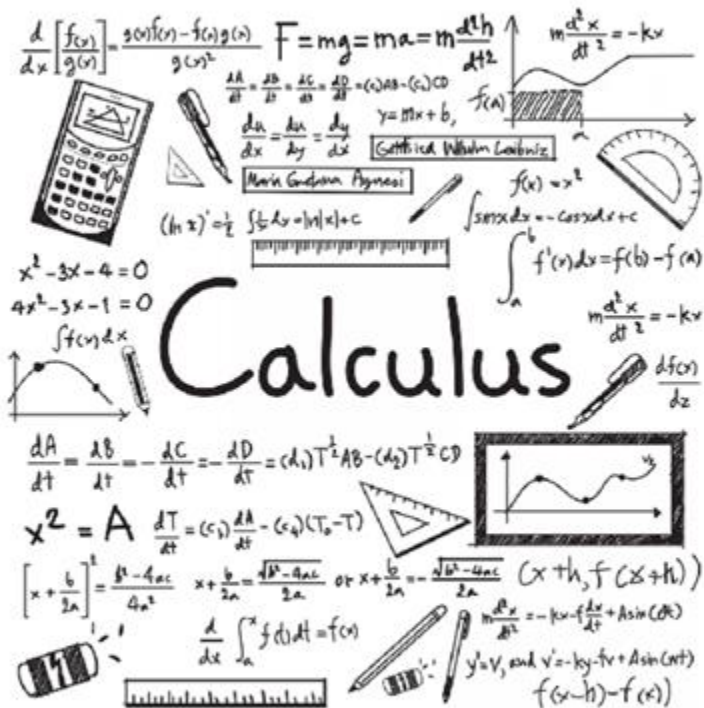
5-d



```
[[[ [...  
    ...  
    ... ]]]
```

A batch of videos
(batch-size x time x
width x height x
channels)

Matrix



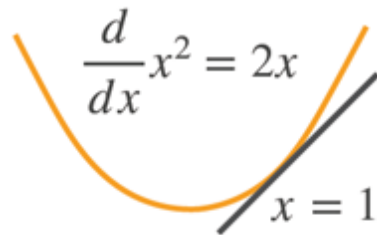
Review Scalar Derivative

y	a	x^n	$\exp(x)$	$\log(x)$	$\sin(x)$
$\frac{dy}{dx}$	0	nx^{n-1}	$\exp(x)$	$\frac{1}{x}$	$\cos(x)$

a is not a function of x

y	$u + v$	uv	$y = f(u), u = g(x)$
$\frac{dy}{dx}$	$\frac{du}{dx} + \frac{dv}{dx}$	$\frac{du}{dx}v + \frac{dv}{dx}u$	$\frac{dy}{du} \frac{du}{dx}$

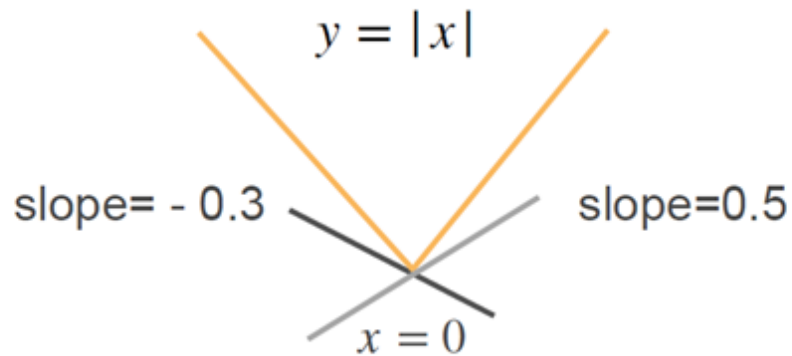
Derivative is the slope of the tangent line



The slope of the tangent line is 2

Subderivative

- Extend derivative to non-differentiable cases



$$\frac{\partial |x|}{\partial x} = \begin{cases} 1 & \text{if } x > 0 \\ -1 & \text{if } x < 0 \\ a & \text{if } x = 0, \quad a \in [-1, 1] \end{cases}$$

Another example:

$$\frac{\partial}{\partial x} \max(x, 0) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x < 0 \\ a & \text{if } x = 0, \quad a \in [0, 1] \end{cases}$$

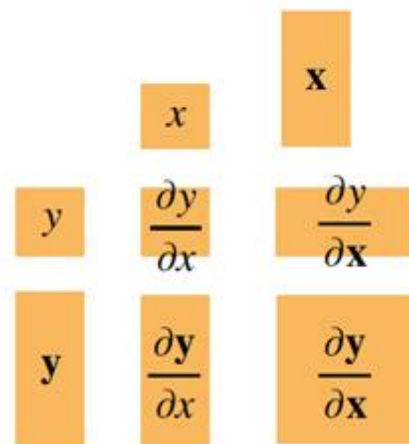
Gradients

- Generalize derivatives into vectors

		Scalar	Vector
		x	$\mathbf{x}$
Scalar	y	$\frac{\partial y}{\partial x}$	$\frac{\partial y}{\partial \mathbf{x}}$
Vector	$\mathbf{y}$	$\frac{\partial \mathbf{y}}{\partial x}$	$\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$

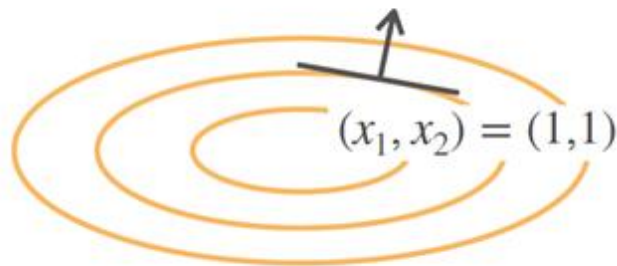
$$\partial y / \partial \mathbf{x}$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad \frac{\partial y}{\partial \mathbf{x}} = \left[\frac{\partial y}{\partial x_1}, \frac{\partial y}{\partial x_2}, \dots, \frac{\partial y}{\partial x_n} \right]$$



$$\frac{\partial}{\partial \mathbf{x}} x_1^2 + 2x_2^2 = [2x_1, 4x_2]$$

Direction (2, 4), perpendicular to the contour lines



Examples

y	a	au	$\text{sum}(\mathbf{x})$	$\ \mathbf{x}\ ^2$
$\frac{\partial y}{\partial \mathbf{x}}$	$\mathbf{0}^T$	$a \frac{\partial u}{\partial \mathbf{x}}$	$\mathbf{1}^T$	$2\mathbf{x}^T$

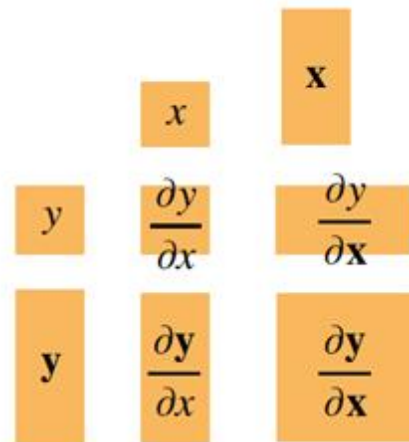
a is not a function of $\mathbf{x}$

$\mathbf{0}$ and $\mathbf{1}$ are vectors

y	$u + v$	uv	$\langle \mathbf{u}, \mathbf{v} \rangle$
$\frac{\partial y}{\partial \mathbf{x}}$	$\frac{\partial u}{\partial \mathbf{x}} + \frac{\partial v}{\partial \mathbf{x}}$	$\frac{\partial u}{\partial \mathbf{x}} v + \frac{\partial v}{\partial \mathbf{x}} u$	$\mathbf{u}^T \frac{\partial \mathbf{v}}{\partial \mathbf{x}} + \mathbf{v}^T \frac{\partial \mathbf{u}}{\partial \mathbf{x}}$

$$\partial \mathbf{y} / \partial x$$

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} \quad \frac{\partial \mathbf{y}}{\partial x} = \begin{bmatrix} \frac{\partial y_1}{\partial x} \\ \frac{\partial y_2}{\partial x} \\ \vdots \\ \frac{\partial y_m}{\partial x} \end{bmatrix}$$



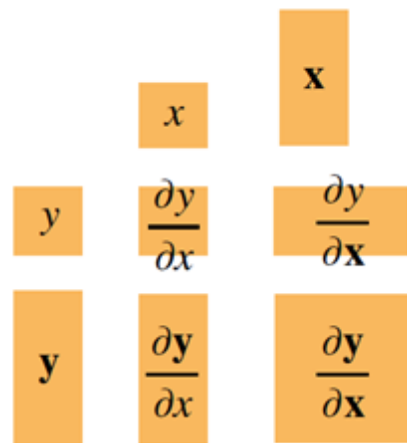
$\partial y / \partial \mathbf{x}$ is a row vector, while $\partial \mathbf{y} / \partial x$ is a column vector

It is called numerator-layout notation. The reversed version is called denominator-layout notation

$$\partial \mathbf{y} / \partial \mathbf{x}$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial \mathbf{x}} \\ \frac{\partial y_2}{\partial \mathbf{x}} \\ \vdots \\ \frac{\partial y_m}{\partial \mathbf{x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1}, \frac{\partial y_1}{\partial x_2}, \dots, \frac{\partial y_1}{\partial x_n} \\ \frac{\partial y_2}{\partial x_1}, \frac{\partial y_2}{\partial x_2}, \dots, \frac{\partial y_2}{\partial x_n} \\ \vdots \\ \frac{\partial y_m}{\partial x_1}, \frac{\partial y_m}{\partial x_2}, \dots, \frac{\partial y_m}{\partial x_n} \end{bmatrix}$$



Examples

y	a	x	Ax	$x^T A$
$\frac{\partial y}{\partial x}$	0	I	A	A^T

$$x \in \mathbb{R}^n, \quad y \in \mathbb{R}^m, \quad \frac{\partial y}{\partial x} \in \mathbb{R}^{m \times n}$$

a , a and A are not functions of x

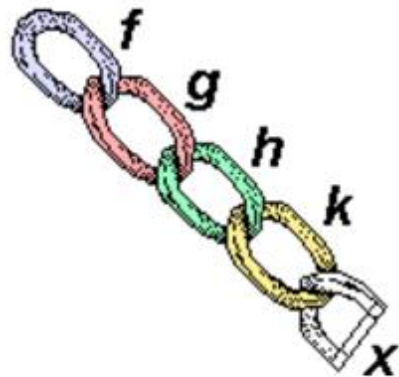
0 and I are matrices

y	au	Au	$u + v$
$\frac{\partial y}{\partial x}$	$a \frac{\partial u}{\partial x}$	$A \frac{\partial u}{\partial x}$	$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial x}$

Generalize to Matrices

		Scalar	Vector	Matrix
		x (1,)	$\mathbf{x}$ (n,1)	$\mathbf{X}$ (n,k)
Scalar	y (1,)	$\frac{\partial y}{\partial x}$ (1,)	$\frac{\partial y}{\partial \mathbf{x}}$ (1,n)	$\frac{\partial y}{\partial \mathbf{X}}$ (k,n)
Vector	$\mathbf{y}$ (m,1)	$\frac{\partial \mathbf{y}}{\partial x}$ (m,1)	$\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ (m,n)	$\frac{\partial \mathbf{y}}{\partial \mathbf{X}}$ (m,k,n)
Matrix	$\mathbf{Y}$ (m,l)	$\frac{\partial \mathbf{Y}}{\partial x}$ (m,l)	$\frac{\partial \mathbf{Y}}{\partial \mathbf{x}}$ (m,l,n)	$\frac{\partial \mathbf{Y}}{\partial \mathbf{X}}$ (m,l,k,n)

Chain Rule



Generalize to Vectors

- Chain rule for scalars:

$$y = f(u), u = g(x) \quad \frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \frac{\partial u}{\partial x}$$

- Generalize to vectors straightforwardly

$$\frac{\partial y}{\partial \mathbf{x}} = \frac{\partial y}{\partial u} \frac{\partial u}{\partial \mathbf{x}}$$

$$(1, n) \quad (1,) \quad (1, n)$$

$$\frac{\partial y}{\partial \mathbf{x}} = \frac{\partial y}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{x}}$$

$$(1, n) \quad (1, k) \quad (k, n)$$

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{x}}$$

$$(m, n) \quad (m, k) \quad (k, n)$$

Example 1

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{x}}$$

Assume $\mathbf{x}, \mathbf{w} \in \mathbb{R}^n$, $y \in \mathbb{R}$

$$z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$$

Compute $\frac{\partial z}{\partial \mathbf{w}}$

$$\begin{aligned}\frac{\partial z}{\partial \mathbf{w}} &= \frac{\partial z}{\partial b} \frac{\partial b}{\partial a} \frac{\partial a}{\partial \mathbf{w}} \\ &= \frac{\partial b^2}{\partial b} \frac{\partial a - y}{\partial a} \frac{\partial \langle \mathbf{x}, \mathbf{w} \rangle}{\partial \mathbf{w}} \\ &= 2b \cdot 1 \cdot \mathbf{x}^T \\ &= 2 (\langle \mathbf{x}, \mathbf{w} \rangle - y) \mathbf{x}^T\end{aligned}$$

Decompose

$$a = \langle \mathbf{x}, \mathbf{w} \rangle$$

$$b = a - y$$

$$z = b^2$$

Example 2

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{x}}$$

Assume $\mathbf{X} \in \mathbb{R}^{m \times n}$, $\mathbf{w} \in \mathbb{R}^n$, $\mathbf{y} \in \mathbb{R}^m$

$$z = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$$

Compute $\frac{\partial z}{\partial \mathbf{w}}$

$$\begin{aligned} \frac{\partial z}{\partial \mathbf{w}} &= \frac{\partial z}{\partial \mathbf{b}} \frac{\partial \mathbf{b}}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{w}} \\ &= \frac{\partial \|\mathbf{b}\|^2}{\partial \mathbf{b}} \frac{\partial \mathbf{a} - \mathbf{y}}{\partial \mathbf{a}} \frac{\partial \mathbf{X}\mathbf{w}}{\partial \mathbf{w}} \\ &= 2\mathbf{b}^T \times \mathbf{I} \times \mathbf{X} \\ &= 2(\mathbf{X}\mathbf{w} - \mathbf{y})^T \mathbf{X} \end{aligned}$$

Decompose

$$\begin{aligned} \mathbf{a} &= \mathbf{X}\mathbf{w} \\ \mathbf{b} &= \mathbf{a} - \mathbf{y} \\ z &= \|\mathbf{b}\|^2 \end{aligned}$$

Auto Differentiation



Auto Differentiation (AD)

- AD evaluates gradients of a function specified by a program at given values
- AD differs to
 - Symbolic differentiation

In[1]:= D[4 x³ + x² + 3, x]

Out[1]= 2 x + 12 x²

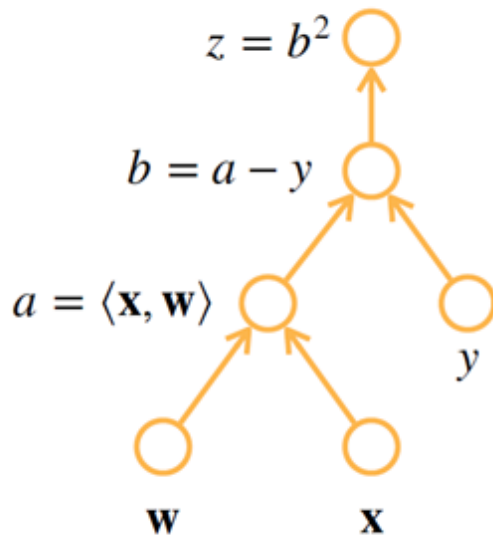
- Numerical differentiation

$$\frac{\partial f(x)}{\partial x} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Computation Graph

- Decompose into primitive operations
- Build a directed acyclic graph to present the computation

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



Computation Graph

- Decompose into primitive operations
- Build a directed acyclic graph to present the computation
- Build explicitly
 - Tensorflow/Theano/MXNet
- Build implicitly though tracing
 - PyTorch/MXNet

```
from mxnet import autograd, nd
```

```
with autograd.record():  
    a = nd.ones((2,1))  
    b = nd.ones((2,1))  
    c = 2 * a + b
```

Two Modes

- By chain rule $\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u_n} \frac{\partial u_n}{\partial u_{n-1}} \dots \frac{\partial u_2}{\partial u_1} \frac{\partial u_1}{\partial x}$

- Forward accumulation

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u_n} \left(\frac{\partial u_n}{\partial u_{n-1}} \left(\dots \left(\frac{\partial u_2}{\partial u_1} \frac{\partial u_1}{\partial x} \right) \right) \right)$$

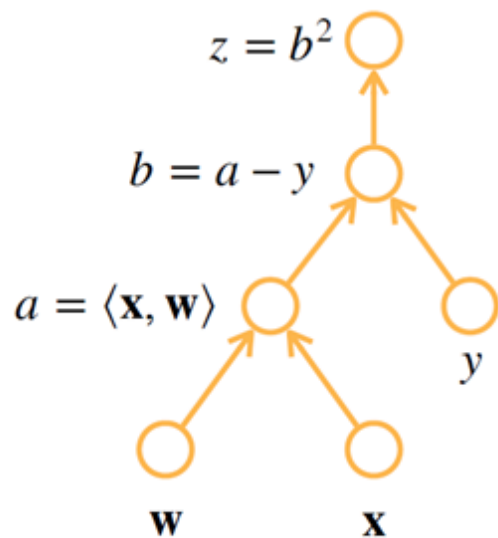
- Reverse accumulation (a.k.a Backpropagation)

$$\frac{\partial y}{\partial x} = \left(\left(\left(\frac{\partial y}{\partial u_n} \frac{\partial u_n}{\partial u_{n-1}} \right) \dots \right) \frac{\partial u_2}{\partial u_1} \right) \frac{\partial u_1}{\partial x}$$

Reverse Accumulation

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$

Forward

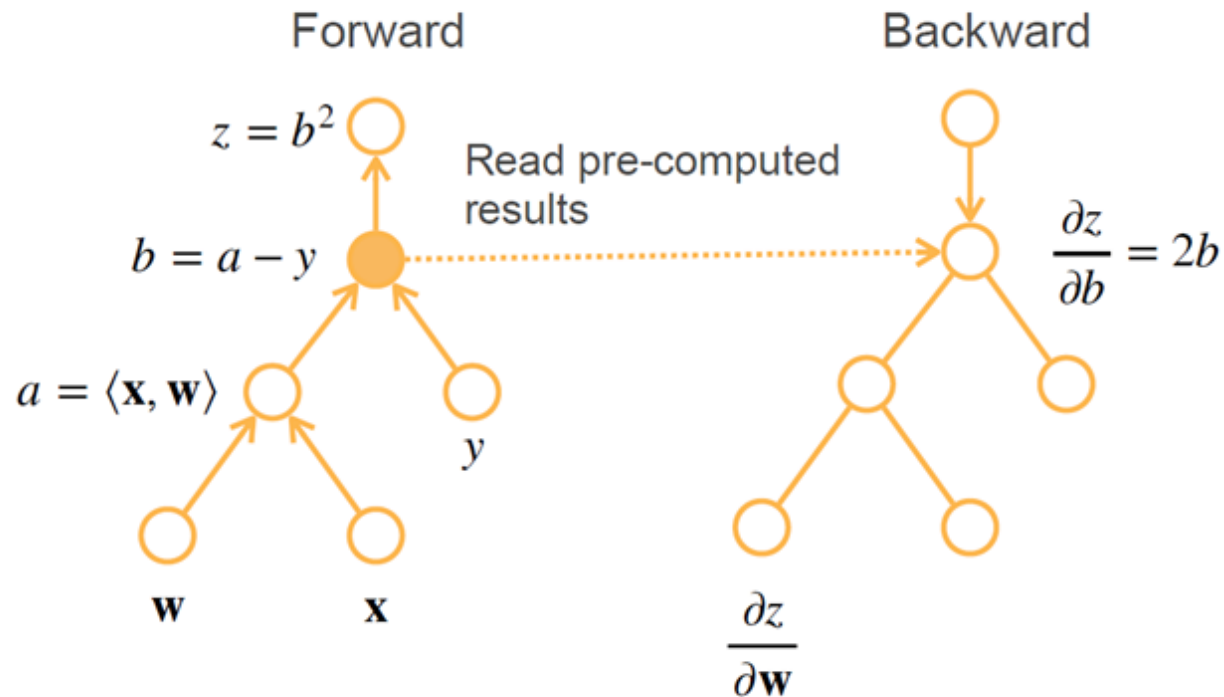


Backward



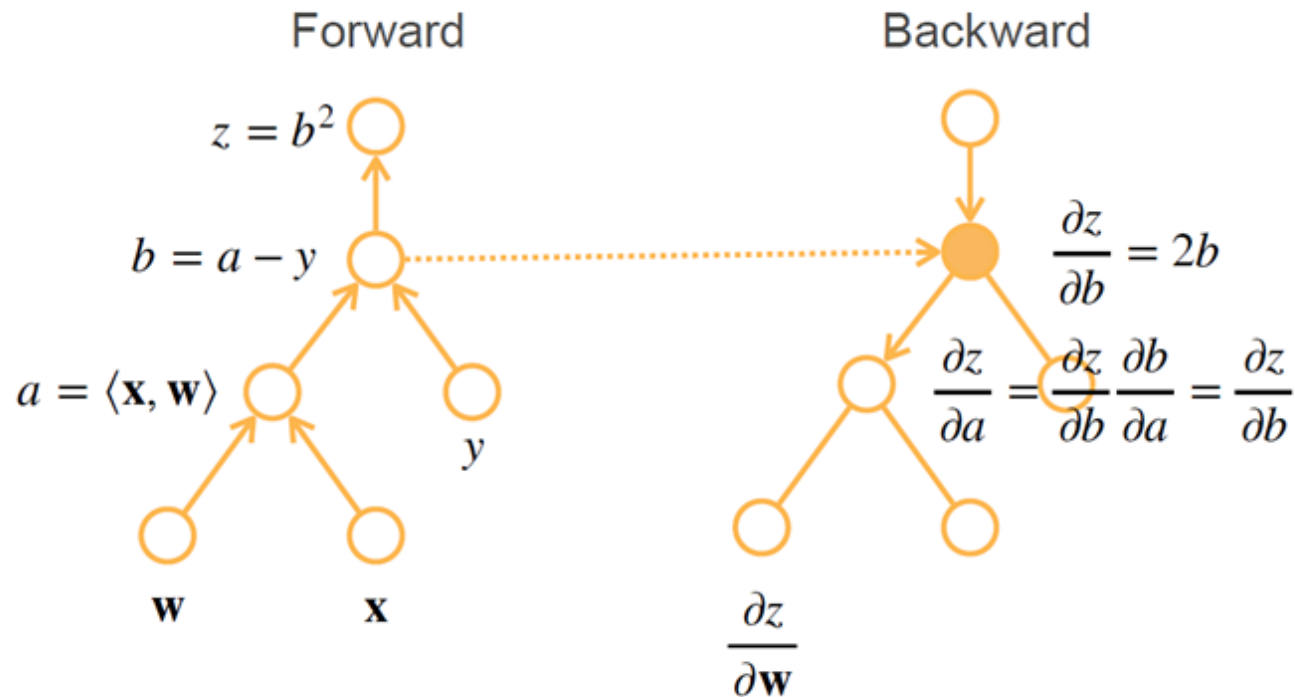
Reverse Accumulation

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



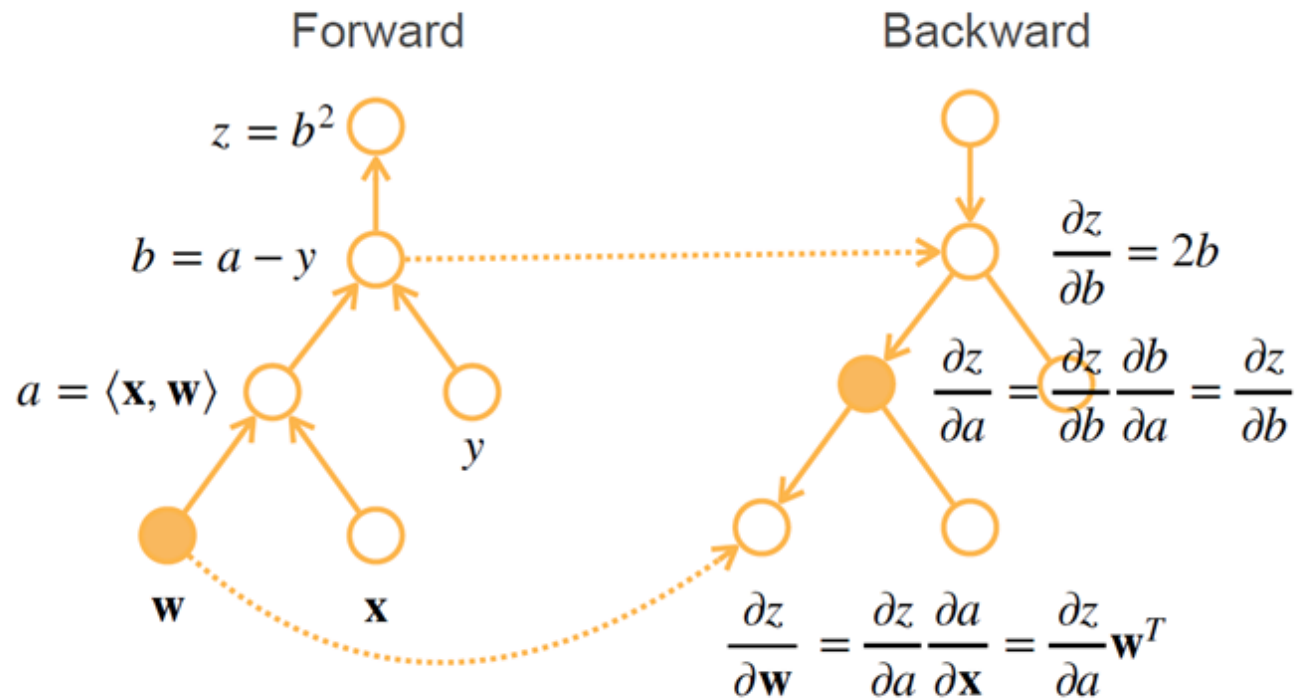
Reverse Accumulation

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



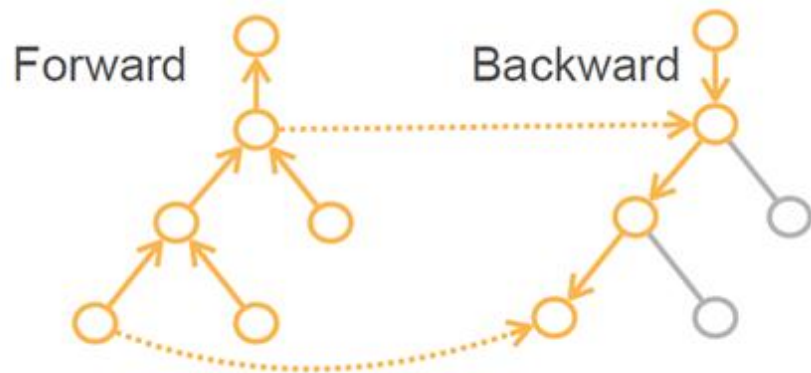
Reverse Accumulation

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$

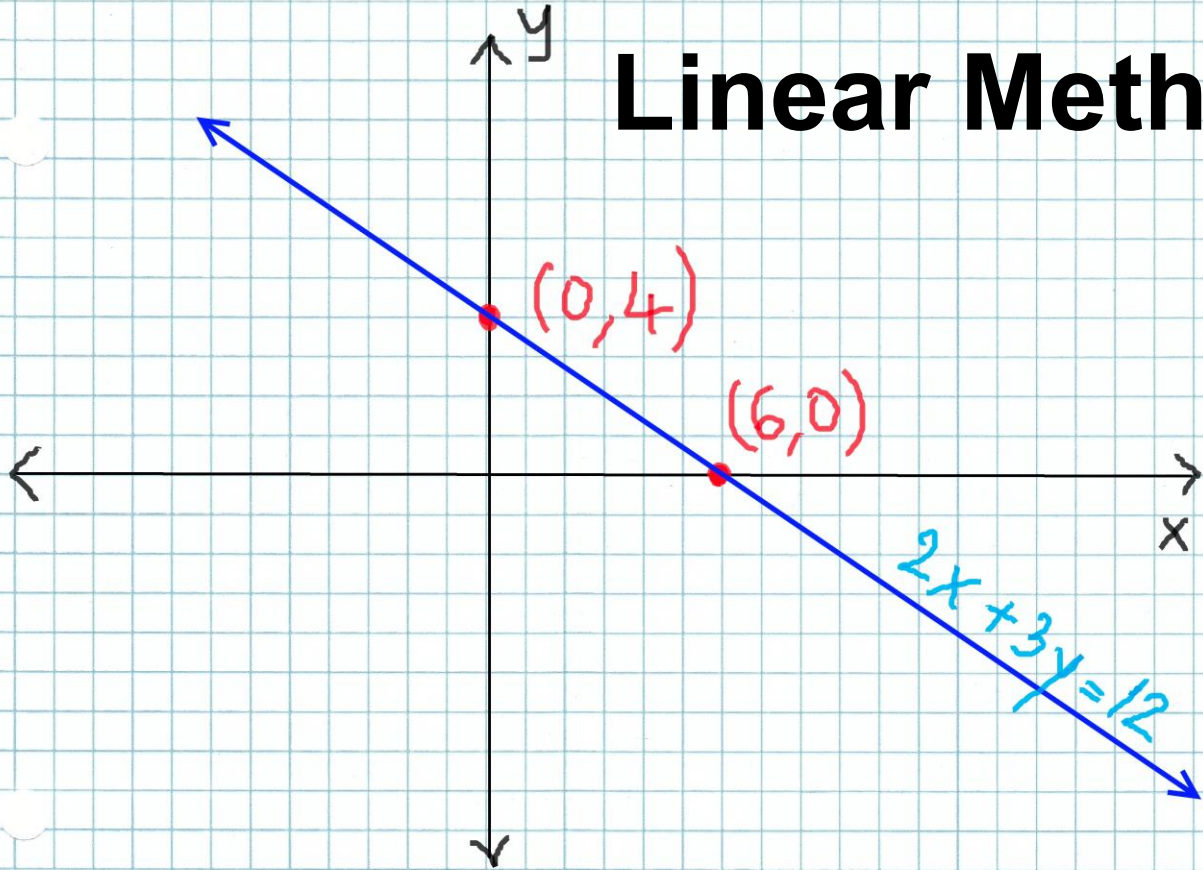


Reverse Accumulation Summary

- Build a computation graph
- Forward: Evaluate the graph, store intermediate results
- Backward: Evaluate the graph in a reversed order
 - Eliminate paths not needed



Linear Methods



House Buying 101

Pick a house, take a tour,
and read facts

Estimate its price, bid

Listing price
from agent

\$5,498,000

Price

7

Beds

5

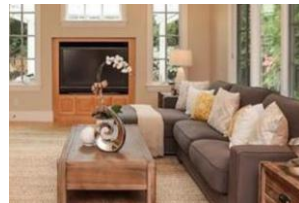
Baths

4,865 Sq. Ft.

\$1130 / Sq. Ft.

Redfin Estimate: \$5,390,037 On Redfin: 15 days

Predicted
sale price



Virtual Tour

- [Branded Virtual Tour](#)
- [Virtual Tour \(External Link\)](#)

Parking Information

- Garage (Minimum): 2
- Garage (Maximum): 2
- Parking Description: Attached Garage, On Street
- Garage Spaces: 2

Interior Features

Bedroom Information

- # of Bedrooms (Minimum): 7
- # of Bedrooms (Maximum): 7

Multi-Unit Information

- # of Stories: 2

School Information

- Elementary School: El Carmelo Elementa
- Elementary School District: Palo Alto Uni
- Middle School: Jane Lathrop Stanford Mix
- High School: Palo Alto High
- High School District: Palo Alto Unified

- Kitchen Description: Countertop (Granite Dishwasher, Garbage Disposal, Hood Ove Island with Sink, Microwave, Oven Range

A Simplified Model

7 Beds	5 Baths	4,865 Sq. Ft. \$1130 / Sq. Ft.
Estimate: \$5,390,037		On Redfin: 15 days

Assumption 1

The key factors impacting the prices are

#Beds, #Baths, Living Sqft, denoted by x_1, x_2, x_3

Assumption 2

The sale price is a weighted sum over the key factors

$$y = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Weights and bias are determined later

Linear Model

Given n -dimensional inputs $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$

Linear model has a n -dimensional weight and a bias

$$\mathbf{w} = [w_1, w_2, \dots, w_n]^T, b$$

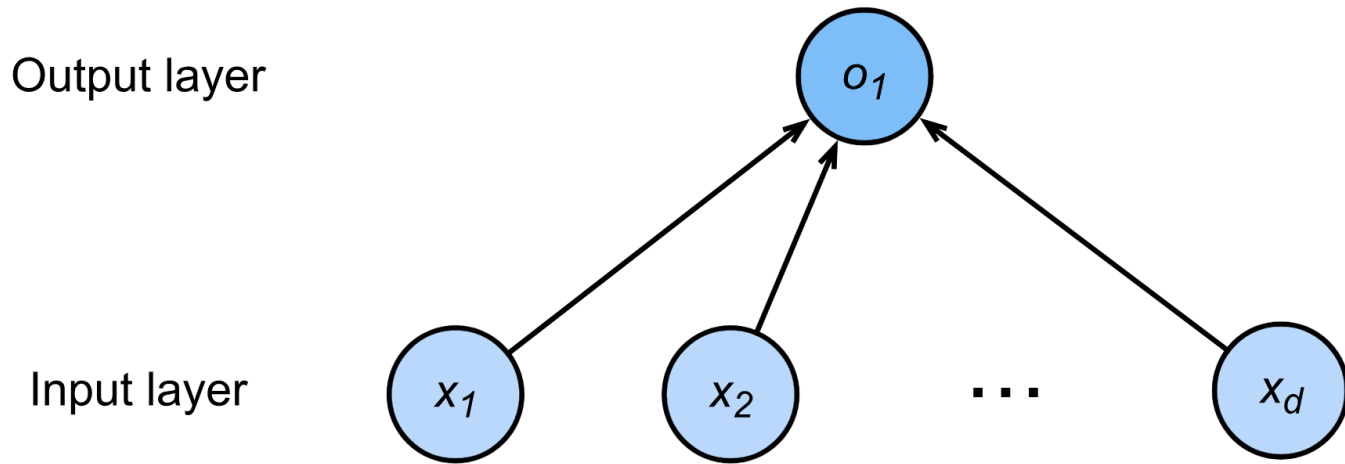
The output is a weighted sum of the inputs

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Vectorized version

$$y = \langle \mathbf{w}, \mathbf{x} \rangle + b$$

Linear Model as a Single-layer Neural Network



We can stack multiple layers to get deep neural networks

Measure Estimation Quality

Compare the true value vs the estimated value

Real sale price vs estimated house price

Let y the true value, and $\hat{y}$ the estimated value, we can compare the loss

$$\ell(y, \hat{y}) = (y - \hat{y})^2$$

It is called squared loss

Training Data

Collect multiple data points to fit parameters

Houses sold in the last 6 months

It is called the training data

The more the better

Assume n examples

$$\mathbf{X} = [\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_n]^T \quad \mathbf{y} = [y_0, y_1, \dots, y_n]^T$$

Learn Parameters

Training loss

$$\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \langle \mathbf{x}_i, \mathbf{w} \rangle - b)^2 = \frac{1}{n} \| \mathbf{y} - \mathbf{X}\mathbf{w} - b \|^2$$

Minimize loss to learn parameters

$$\mathbf{w}^*, \mathbf{b}^* = \underset{\mathbf{w}, b}{\operatorname{argmin}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}, b)$$

Closed-form Solution

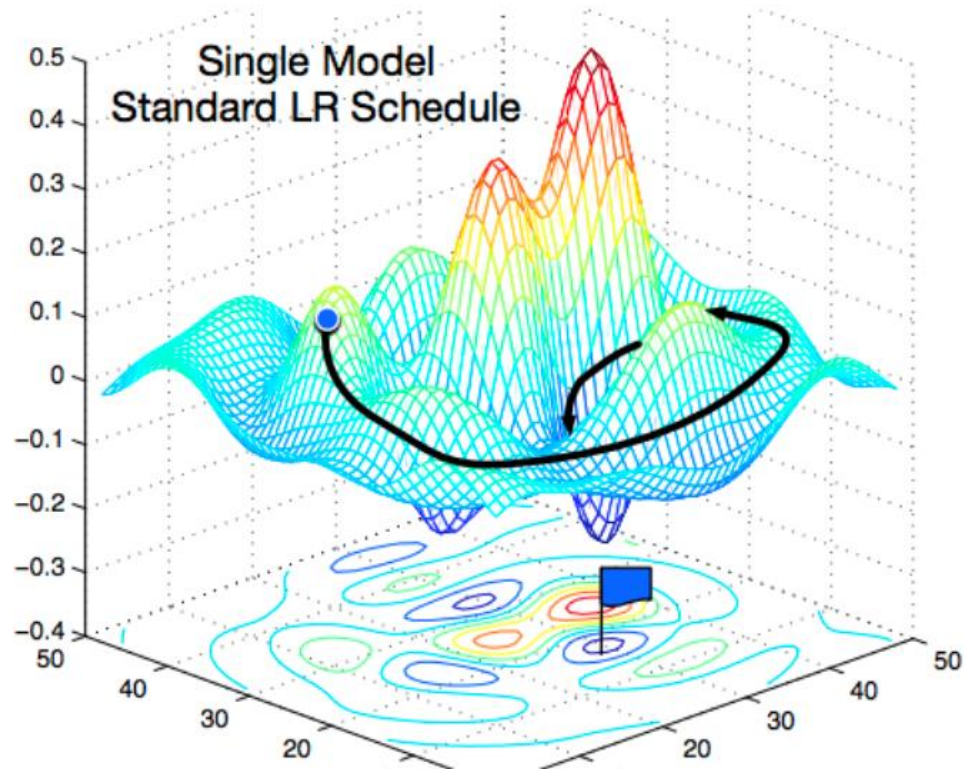
Add bias into weights by $\mathbf{X} \leftarrow [\mathbf{X}, \mathbf{1}] \mathbf{w} \leftarrow \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$

$$\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 \quad \frac{\partial}{\partial \mathbf{w}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \frac{2}{n} (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{X}$$

Loss is convex, so the optimal solutions satisfies

$$\begin{aligned} \frac{\partial}{\partial \mathbf{w}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) &= 0 \\ \Leftrightarrow \frac{2}{n} (\mathbf{y} - \mathbf{X}\mathbf{w})^T \mathbf{X} &= 0 \\ \Leftrightarrow \mathbf{w}^* &= (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X} \mathbf{y} \end{aligned}$$

Basic Optimization



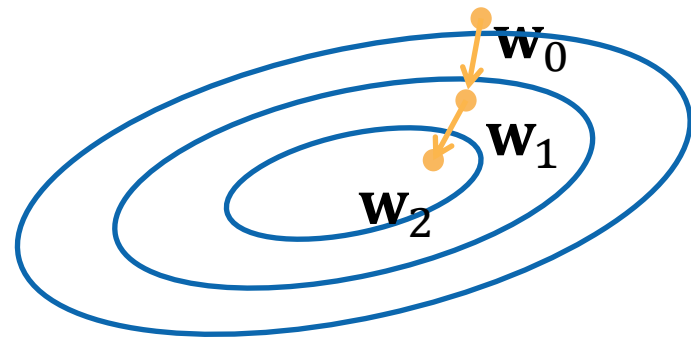
Gradient Descent

Choose a starting point $\mathbf{w}_0$

Repeat to update the weight $t=1,2,3$

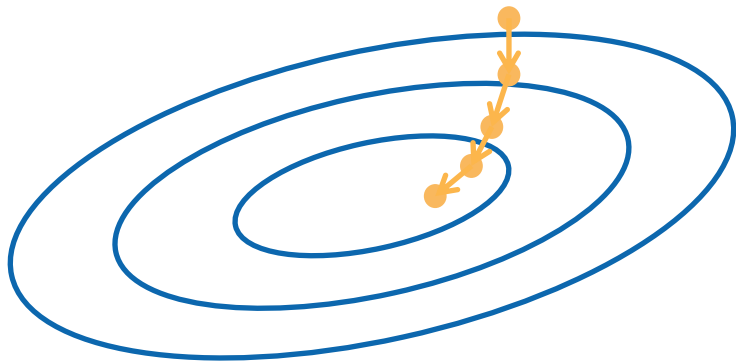
$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \frac{\partial \ell}{\partial \mathbf{w}_{t-1}}$$

- Gradient: a direction that increases the value
- Learning rate: a hyper-parameter specifies the step length

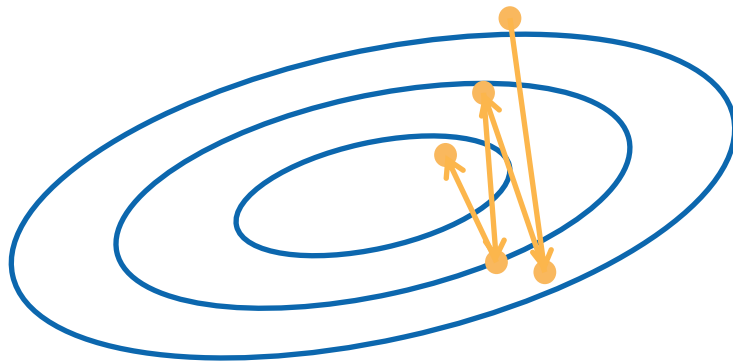


Choose a Learning Rate

Not too small



Not too big



Mini-batch Stochastic Gradient Descent (SGD)

Computing the gradient over the whole training data is too expensive

- Takes minutes to hours for DNN models

Randomly sample b examples $i_1, i_2, \dots, i_b$ to approximate the loss

$$\frac{1}{b} \sum_{i \in I_b} \ell(\mathbf{x}_i, y_i, \mathbf{w})$$

- b is the batch size, another important hyperparameters

Choose a Batch Size

Not too small

Workload is too small, hard to fully utilize computation resources

Not too big

Memory issue
Waste computation, e.g. when all x_i are identical

Summary

Problem: estimate a real value

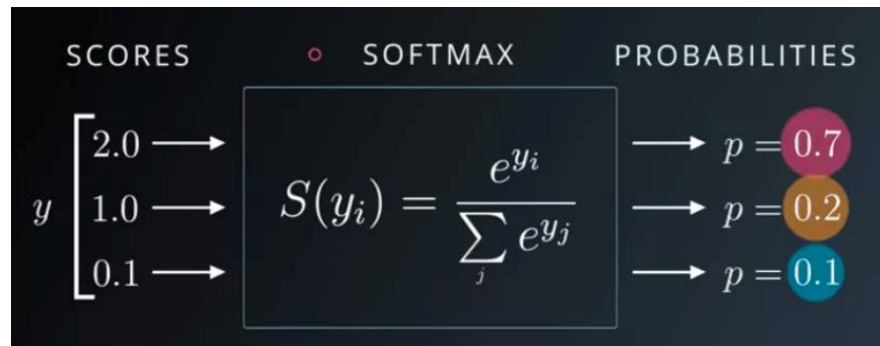
Model: $y = \langle \mathbf{w}, \mathbf{x} \rangle + b$

Loss: square loss $\ell(y, \hat{y}) = (y - \hat{y})^2$

Learn by mini-batch SGD

- Choose a starting point
- Repeat
 - Compute gradient
 - Update parameters

Softmax Regression



Regression vs Classification

Regression estimates a continuous value

Classification predicts a discrete category

MNIST: classify hand-written digits
(10 classes)

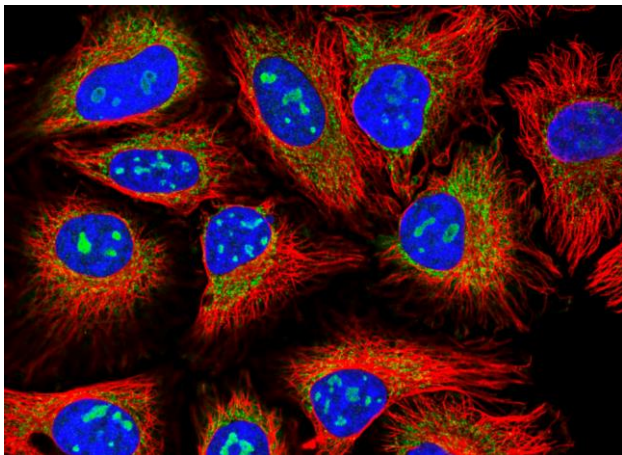


ImageNet: classify nature objects
(1000 classes)



Various Classification Tasks at Kaggle

Classify human protein microscope images into 28 categories



- 0. Nucleoplasm
- 1. Nuclear membrane
- 2. Nucleoli
- 3. Nucleoli fibrillar
- 4. Nuclear speckles
- 5. Nuclear bodies
- 6. Endoplasmic reticu
- 7. Golgi apparatus
- 8. Peroxisomes
- 9. Endosomes
- 10. Lysosomes
- 11. Intermediate fila
- 12. Actin filaments
- 13. Focal adhesion si
- 14. Microtubules
- 15. Microtubule ends
- 16. Cytokinetic bridg

<https://www.kaggle.com/c/human-protein-atlas-image-classification>

Various Classification Tasks at Kaggle

Classify malware into 9 categories



<https://www.kaggle.com/c/malware-classification>

Various Classification Tasks at Kaggle

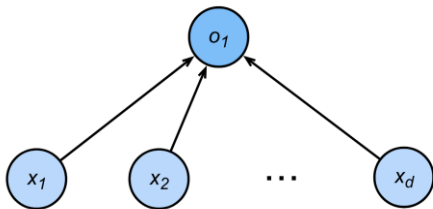
Classify toxic Wikipedia comments into 7 categories

comment_text	toxic	severe_toxic	obsc
Explanation\nWhy the edits made under my usern...	0	0	0
D'aww! He matches this background colour I'm S...	0	0	0
Hey man, I'm really not trying to edit war. It...	0	0	0
"\nMore\nI can't make any real suggestions on ...	0	0	0
You, sir, are my hero. Any chance you remember...	0	0	0

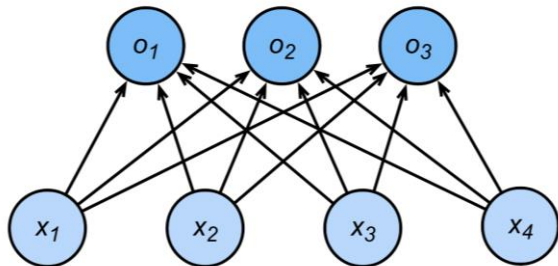
<https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge>

From Regression to Multi-class Classification

Single continuous output



Output a confidence score for each category



- Predict the category

$$\operatorname{argmax}_i(o_1, o_2, o_3)$$

- *max* is not differentiable
- define the loss function

Softmax

$$\text{softmax}([x_1, x_2, \dots, x_n]^T) = \left[\frac{e^{x_1}}{\sum_{i=1}^n e^{x_i}}, \frac{e^{x_2}}{\sum_{i=1}^n e^{x_i}}, \dots, \frac{e^{x_n}}{\sum_{i=1}^n e^{x_i}} \right]$$

- Use exp to get positive numbers
- Divide by the sum to obtain a probability distribution

Example: given scores [1, -1, 2]

softmax: [0.26, 0.04, 0.7]

Softmax Gradient

$$\text{softmax}([x_1, x_2, \dots, x_n]^T) = \left[\frac{e^{x_1}}{\sum_{i=1}^n e^{x_i}}, \frac{e^{x_2}}{\sum_{i=1}^n e^{x_i}}, \dots, \frac{e^{x_n}}{\sum_{i=1}^n e^{x_i}} \right]$$

$$\frac{\partial}{\partial \mathbf{x}} \text{softmax}(\mathbf{x})$$

<https://aimatters.wordpress.com/2019/06/17/the-softmax-function-derivative/>

Use Square Loss?

One-hot encoding of label k

$$\mathbf{y} = [y_1, y_2, \dots, y_n]^T, y_i = \begin{cases} 1 & \text{if } i = k \\ 0 & \text{otherwise} \end{cases}$$

We want the softmax output close to $\mathbf{y}$

Use square loss for $\mathbf{y} = [0, 0, 1]$

Softmax output	Loss
[0.3, 0, 0.7]	0.18
[0.17, 0.17, 0.66]	0.173

Cross Entropy Loss

Both one-hot encoding and softmax output are probability distribution

Cross entropy is commonly used to compare distributions

$$H(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$

- If $i = k$ (*ground truth*) , then

$$H(\mathbf{y}, \hat{\mathbf{y}}) = -\log(\hat{y}_i)$$

Linear Model vs Logistic Regression

Problem	single value regression	k-class classification
Model	$\langle \mathbf{w}, \mathbf{x} \rangle + b$ $\mathbf{w} \in \mathbb{R}^n, b \in \mathbb{R}$	$\text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b})$ $\mathbf{W} \in \mathbb{R}^{k \times n}, \mathbf{b} \in \mathbb{R}^k$
Loss	Squared loss	Cross-entropy loss

The End